



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo Práctico

24 de septiembre de 2026

Aprendizaje Automático I

Grupo 11

Integrante	LU	Correo electrónico
Bravo, Iván Agustín	786/23	bravoivanagustin@gmail.com
Kiss, María Delfina	72/23	kissdelfina@gmail.com
Lützow Holm, Eric	323/12	elholm90@gmail.com
Mendoza, Juan Cruz	004/01	email4@dominio.com
Rostan, Marcos Andrés	589/23	rostanmarcos@gmail.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón Cero + Infinito)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Conmutador: (+54 11) 5285-9721 / 5285-7400

<https://dc.uba.ar>

1. Introducción

En este trabajo práctico evaluamos distintos modelos de aprendizaje automático para determinar si una muestra de células tiene pronóstico de crecimiento anómalo o descontrolado, condición conocida como *hiperplasia*. El dataset consta de 500 instancias provenientes de pacientes con lesiones pretumorales, en las que se midió la expresión de 200 genes. Cada instancia está etiquetada como verdadero (buen pronóstico) si no hubo indicios de una nueva *hiperplasia*, o falso (mal pronóstico) en caso contrario.

Probamos diferentes algoritmos con distintas configuraciones de hiperparámetros, con el fin de comparar su comportamiento en esta tarea de clasificación: árboles de decisión, K vecinos más cercanos (KNN), support vector machines (SVM), análisis discriminante lineal (LDA) y Naive Bayes. La implementación se realizó en Python.¹

2. Separación de datos

Para estimar la performance del modelo final dividimos el dataset en dos particiones: un conjunto de desarrollo, sobre el cual entrenamos y seleccionamos los modelos, y un conjunto de control, para evaluar al final del trabajo la configuración elegida. Siguiendo el estándar habitual, destinamos el 15 % de las instancias al conjunto de control.

La partición debe ser representativa del dataset, la proporción de etiquetas tiene que coincidir con la del conjunto de desarrollo, y el conjunto de control no debería presentar propiedades ausentes en el resto de los datos, ni a la inversa.

Del total, 359 instancias (71,8 %) corresponden a la etiqueta 0 (mal pronóstico) y 141 (28,2 %) a la etiqueta 1 (buen pronóstico). Para preservar esas proporciones aplicamos muestreo estratificado: agrupamos las instancias por etiqueta y extrajimos una muestra aleatoria del 15 % dentro de cada grupo. Lo implementamos con los métodos `groupby` y `sample` de Pandas, fijando una semilla para que la partición sea reproducible. El resultado son $0,15 \cdot 359 \approx 54$ instancias de etiqueta 0 y $0,15 \cdot 141 \approx 21$ de etiqueta 1, lo que deja un conjunto de control de 75 instancias y un conjunto de desarrollo de 425.

Con la separación establecida, construimos los modelos.

3. Construcción de modelos

Buscamos construir y evaluar modelos de árboles de decisión, obteniendo una estimación realista de su performance. Como primer paso separamos el conjunto de desarrollo en entrenamiento y validación de manera proporcional a sus etiquetas, usando `train_test_split` de `sklearn` estratificado, y nos quedamos con un 0,8 de entrenamiento y 0,2 de validación. Sobre esa partición entrenamos un árbol con altura máxima 3 y el resto de los hiperparámetros en sus valores por defecto.

De este árbol obtuvimos un accuracy de 0.6471 sobre el subconjunto de validación. Ese valor sale de una única partición de 85 instancias, por lo que depende bastante de cuáles hayan caído del lado de validación. Para obtener una estimación más estable repetimos la medición con validación cruzada sobre el conjunto de desarrollo completo, con un K-fold estratificado de 5 particiones de manera que cada fold conserve la proporción de etiquetas. Para cada fold calculamos Accuracy, Area Under the Precision-Recall Curve (AUPRC) y Area Under the Receiver Operating Characteristic Curve (AUCROC), y además el score global utilizando únicamente los folds de validación. Los resultados se pueden observar en la Tabla 1.

¹El notebook con el código completo se puede consultar aquí. En este informe incluimos solo el código mínimo necesario para ilustrar los pasos más importantes.

Permutación	Acc (train)	Acc (val)	AUPRC (train)	AUPRC (val)	AUCROC (train)	AUCROC (val)
1	0.8265	0.6000	0.7153	0.3265	0.8626	0.5697
2	0.8265	0.7412	0.6862	0.4213	0.8037	0.6411
3	0.8353	0.6235	0.6981	0.2895	0.8506	0.4754
4	0.8265	0.6471	0.6900	0.3594	0.8429	0.6117
5	0.7971	0.6824	0.6491	0.3574	0.8538	0.5683
Promedios	0.8224	0.6588	0.6878	0.3508	0.8427	0.5732
Global	(NO)	0.6588	(NO)	0.3404	(NO)	0.5610

Tabla 1: Métricas de un árbol de altura máxima 3 con criterio Gini, por fold de la validación cruzada estratificada de 5 particiones sobre el conjunto de desarrollo. Global se calcula sobre las predicciones acumuladas de los 5 folds de validación

Para interpretar estos valores necesitamos un punto de comparación, así que evaluamos con la misma validación cruzada un clasificador que ignora los atributos y predice siempre la clase mayoritaria. Alcanza un accuracy de 0.7176, un AUPRC de 0.2824 que no es otra cosa que la proporción de positivos, y un AUCROC de 0.5000 porque asigna el mismo score a todas las instancias.

Luego exploramos 6 combinaciones de altura máxima y criterio de corte, manteniendo validación cruzada con $K = 5$ y generando las combinaciones con `ParameterGrid` de scikit-learn. Probamos alturas máximas de 3, 5 e infinita, con criterio de corte Gini y Entropía. Reutilizamos las mismas particiones para que los valores sean comparables, lo que se verifica en que la fila de altura 3 con Gini reproduce exactamente el resultado de la Tabla 1. Los resultados se muestran en la Tabla 2.

Altura máxima	Criterio de corte	Accuracy (training)	Accuracy (validación)
3	Gini	0.8224	0.6588
5	Gini	0.9482	0.6447
Infinito	Gini	1.0000	0.6400
3	Entropía	0.8029	0.6541
5	Entropía	0.9253	0.6612
Infinito	Entropía	1.0000	0.5953

Tabla 2: Accuracy promedio en entrenamiento y validación de árboles de decisión, según altura máxima y criterio de corte, con la misma validación cruzada estratificada de 5 particiones sobre el conjunto de desarrollo.

De la Tabla 1 vemos que el modelo base, con altura máxima 3 y criterio Gini, logró un accuracy de validación de 0.6588, por debajo del 0.7176 del clasificador trivial. Su AUCROC de validación es 0.5732 en promedio y 0.5610 global, por encima del 0.5000 de referencia, y su AUPRC de 0.3508 mejora un poco el 0.2824 que ya da predecir siempre la clase mayoritaria. El modelo base entonces casi no captura señal, aunque la brecha con entrenamiento es amplia, 0.8224 contra 0.6588 en accuracy y 0.8427 contra 0.5732 en AUCROC.

En la Tabla 2 se observa el comportamiento típico del overfitting. Al relajar la altura máxima el accuracy de entrenamiento sube hasta 1.0000, es decir que el árbol memoriza las 340 instancias de cada fold, mientras que el de validación no logra mejoras significativas. Con Gini el mejor resultado se da con altura 3 (0.6588) para luego caer a 0.6400 sin restricción, y con Entropía el mejor caso es el árbol de altura 5 (0.6612), empeorando en la configuración sin restricción.

Ninguna de las 6 configuraciones supera el 0.7176 del clasificador trivial, y el rango completo de la columna de validación, que va de 0.5953 a 0.6612, es más angosto que la variación entre folds de una misma configuración, que en la Tabla 1 va de 0.6000 a 0.7412. Por eso no consideramos que ninguna combinación sea preferible a las demás. Tampoco el accuracy es la métrica más adecuada a este problema, porque con casi 71,8 % de instancias negativas premia al modelo que ignora la clase minoritaria, que es justamente la que interesa detectar, y por eso de acá en adelante usamos

AUCROC. En conjunto, las dos tablas muestran que un árbol de decisión no alcanza para estos 200 atributos, lo que motiva explorar otras familias de algoritmos.

4. Comparación de algoritmos

En esta sección se evalúan modelos basados en árboles de decisión, KNN y SVM, explorando distintas combinaciones de hiperparámetros.

4.1. Árboles de decisión

Como hiperparámetros se consideraron la profundidad máxima, la cantidad mínima de instancias necesarias para dividir un nodo, la cantidad mínima de instancias por hoja, el criterio de partición (Gini o Entropía) y la cantidad máxima de atributos a evaluar (probando la raíz cuadrada y el logaritmo del total de variables).

```
parametros_arbol = {"max_depth": [3, 5, 10, 20, None],
                    "min_samples_split": [2, 5, 10, 20],
                    "min_samples_leaf": [1, 5, 10, 15],
                    "criterion": ["gini", "entropy"],
                    "max_features": ["sqrt", "log2", None]}
```

Los mejores hiperparámetros encontrados fueron: 'min_samples_split': 10, 'min_samples_leaf': 15, 'max_features': 'log2', 'max_depth': 5, 'criterion': 'gini', obteniendo un AUCROC score de 0.6740.

4.2. KNN

Para KNN, es importante considerar el escalado de las variables. Al tener escalas muy distintas, aquellas con mayor rango numérico pueden llegar a dominar el cálculo de la distancia. Los hiperparámetros evaluados fueron:

- **n_neighbors**: mientras un k pequeño puede llegar a sobreajustarse al ruido local de cada instancia, un k grande tiende a aproximarse a predecir la clase mayoritaria.
- **weights**: permite asignar mayor peso a los puntos más cercanos.
- **metric**: define la métrica de distancia utilizada para medir la cercanía.

```
parametros_knn = {"knn__n_neighbors": [3, 5, 7, 10, 15, 21, 31, 41, 51, 61, 75],
                  "knn__weights": ["uniform", "distance"],
                  "knn__metric": ["euclidean", "manhattan"],
                  "scaler": [StandardScaler(), None]}
```

Mejores hiperparámetros encontrados para KNN: 'scaler': None, 'knn__weights': 'distance', 'knn__n_neighbors': 21, 'knn__metric': 'manhattan', alcanzando un AUCROC de 0.8474.

4.3. SVM

Para SVM, al igual que en KNN, la escala de las características suele ser un factor crítico. Se exploraron los siguientes hiperparámetros:

- **c**: controla la regularización. Un valor mayor obliga al modelo a tener un margen estricto (lo que puede llevar a *overfitting*), mientras que un valor menor tolera más puntos mal clasificados.

- **kernel**: determina si la frontera de decisión es lineal en el espacio original de las variables, o si se aplica un mapeo (*kernel trick*) para construir un hiperplano en un espacio de mayor dimensión.
- **degree**: aplica únicamente a kernels polinómicos y define el grado máximo del polinomio.
- **gamma**: para los kernels rbf y poly, define el radio de influencia de cada vector de soporte.

Para **gamma** y **C** se suelen probar diferentes órdenes de magnitud. Para **degree**, probar grados mayores a 3 puede generar un espacio de variables enorme y sobreajustar fácilmente con pocas instancias.

```
parametros_svm = [{"svm__kernel": ["linear"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000],
                  "scaler": [StandardScaler(), None]},
                  {"svm__kernel": ["rbf"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000],
                  "svm__gamma": ["scale", 0.0001, 0.001, 0.01, 0.1, 1],
                  "scaler": [StandardScaler(), None]},
                  {"svm__kernel": ["poly"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000],
                  "svm__degree": [2, 3], "svm__gamma": ["scale", 0.0001, 0.001, 0.01, 0.1, 1],
                  "scaler": [StandardScaler(), None]}]
```

Los mejores hiperparámetros encontrados para SVM fueron 'svm__kernel': 'poly', 'svm__gamma': 1, 'svm__degree': 2, 'svm__C': 1000, 'scaler': None, con un AUCROC de 0.8587.

Se observa en la Tabla 3 que las 5 mejores configuraciones obtienen exactamente el mismo AUCROC. Al no aplicar escalado, las variaciones evaluadas en **C** y **gamma** no llegaron a alterar la frontera de decisión ni el rendimiento del modelo en el conjunto de validación.

Índice	Params	Mean Test Score	Std Test Score
29	{'svm__kernel': 'poly', 'svm__gamma': 0.01, 'svm__degree': 2, 'svm__C': 1, 'scaler': None}	0.858743	0.055193
2	{'svm__kernel': 'poly', 'svm__gamma': 1, 'svm__degree': 2, 'svm__C': 1000, 'scaler': None}	0.858743	0.055193
25	{'svm__kernel': 'poly', 'svm__gamma': 0.01, 'svm__degree': 2, 'svm__C': 100, 'scaler': None}	0.858743	0.055193
23	{'svm__kernel': 'poly', 'svm__gamma': 1, 'svm__degree': 2, 'svm__C': 100, 'scaler': None}	0.858743	0.055193
22	{'svm__kernel': 'poly', 'svm__gamma': 0.001, 'svm__degree': 2, 'svm__C': 1000, 'scaler': None}	0.858743	0.055193

Tabla 3: Mejores configuraciones obtenidas mediante búsqueda de hiperparámetros.

Resulta muy llamativo ver que tanto para SVM y KNN, las mejores configuraciones de hiperparámetros según AUCROC fueron aquellas que no utilizaron el Scaler. Esto indica que las features con un rango más grande terminaron siendo aquellas que también tenían mayor poder predictivo.

4.4. LDA y Naive Bayes

Por último, añadimos a la comparación LDA y Naive Bayes.

Fold	LDA	Naive Bayes
1	0.7131	0.7425
2	0.6919	0.7582
3	0.6291	0.7589
4	0.6086	0.6926
5	0.5608	0.7357
Promedio	0.6407	0.7376

Tabla 4: AUCROC por fold y promedio para LDA y Naive Bayes.

Se observa que el AUCROC promedio de LDA resulta ser el más bajo de los modelos evaluados, con un valor de 0.6407. Esto puede deberse a que se está estimando una matriz de covarianza de

200×200 (debido a las 200 variables utilizadas en el entrenamiento) con pocas instancias por fold, lo que resulta en una estimación ruidosa y casi singular. Para mitigar este problema en el futuro, se podría utilizar el hiperparámetro de `shrinkage`, el cual regulariza la matriz de covarianza.

Por otro lado, Naive Bayes, incluso sin haber realizado una búsqueda de hiperparámetros, alcanzó un AUCROC promedio de 0.7376, un desempeño muy competitivo que se acerca a los resultados obtenidos por SVM y KNN. Al asumir independencia entre las variables, este modelo solo estima la media y la varianza para cada gen individualmente, por lo que dichas estimaciones resultan más robustas incluso contando con pocos datos. Uno de los hiperparámetros que se podría ajustar para optimizarlo es `var_smoothing`, el cual añade a todas las estimaciones una fracción de la mayor varianza observada en el conjunto de datos. Esto funciona como una regularización y evita que, si una variable presenta una varianza cercana a cero en el entrenamiento, un valor nuevo en la fase de prueba reciba una probabilidad nula y termine dominando el producto de probabilidades.

5. Evaluación de performance

El mejor modelo obtenido durante el desarrollo fue el basado en Máquinas de Vectores de Soporte (SVM) con los siguientes hiperparámetros: `{'scaler': None, 'svm__kernel': 'poly', 'svm__degree': 2, 'svm__C': 1000, 'svm__gamma': 1}`.

Para aproximar el desempeño de este modelo sobre el conjunto de control (en posesión de los docentes), se entrenó el modelo final utilizando la totalidad de las instancias del conjunto de desarrollo y, posteriormente, se evaluó sobre el conjunto de evaluación que fue separado al inicio del trabajo. De esta forma, se logró una estimación del desempeño final del modelo ante datos no vistos previamente, evitando sobreestimar su capacidad predictiva.

El valor de AUCROC estimado por el modelo final sobre este conjunto de evaluación fue de **0.9189**. Las probabilidades predichas para la clase positiva, correspondientes a cada instancia del conjunto de control, se encuentran adjuntas a este trabajo en el archivo `11_y_pred_control_9189.csv`. Dicho archivo contiene una única columna denominada `output`, tal como fue requerido en la consigna.

6. Conclusiones

7. Discusión

8. Bibliografía consultada